답변에 그래프(등의 추가 데이터) 추가용 메모

> 해당 내용은 임시로, 개발상황에 따라 변경 될 수 있음.
>
> 차트 이외에도 답변 출력때 추가 데이터 출력을 고민한다면 사용 가능하지 않을지
> 혹은 front 통합 때 참고할 수 있지 않을까 생각함

본문

chart출력은 크게 두가지를 생각했음.

1. back에선 데이터만 전송, front에서 데이터를 통해 출력
2. back에서 matplotlib을 통해 chart 이미지까지 생성, 이미지를 front로 넘겨 출력

이미 matplotlib을 통해 그래프를 뽑는 코드는 완성이되어있고, front에서 chart를 구현할 경우 추후 front와의 통합에서 번거로워질 것 같아 후자를 선택
이미지 전송에는 base64를 사용하게 되었다.

plugin
matplotlib으로 생성 후 base64로 인코딩해서 반환 형식 사용

```
    def _generate_chart(self, results: list, p20: float, avg: float) -> str:
        # 위는 matplotlib의 plt출력과 동일, 생략

        buf = io.BytesIO()
        fig.savefig(buf, format="png", dpi=120)
        plt.close(fig)
        buf.seek(0)
        return base64.b64encode(buf.read()).decode("utf-8")
```

기존 simulation의 return부분에 chart 추가

```
        chart_b64 = self._generate_chart(results, p20, avg)
        return {
               # 기존 return 변수들...
            "chart":              chart_b64,
        }
```

```
```

##

agent

- return 추가. 이 때문에 draft를 그냥 return으로 받을 수 없어 다른 파일도 수정이 필요해진다. 아래 참고

```
        return {
            "draft": draft,
            "chart": sim_result.get("chart"),  # ← 추가
            "updated_params": variables,  # ← 누적 반영용 변수 상태 추가
        }
```

##

orchestrator.run()

- 기존 draft 대신 draft+chart 데이터를 넘긴다, 해당 내용 분리가 필요해 raw로 우선 받아, 이후 반환.

```
# ── 에이전트 호출 ──────────────────────
t_agent = time.monotonic()
extra = {"session_vars": session_vars} if domain == "finance" and session_vars else {}

raw = await agent.generate_draft(
question=question,
retry_prompt=retry_prompt,
profile=profile,
**extra,
)
agent_ms = round((time.monotonic() - t_agent) * 1000)

# finance는 dict, 나머지는 문자열 반환
if domain == "finance":
draft = raw["draft"]
chart = raw.get("chart")
else:
draft = raw
chart = None
```

- 최종 return 파트, 둘 모두 "chart": chart if domain == "finance" else None, 를 추가해준다

```
# approved 반환
return {
...
"draft": draft,
```

```
"chart": chart if domain == "finance" else None, # ← 추가
}

# escalated 반환
return {
...
"draft": draft,
"chart": chart if domain == "finance" else None, # ← 추가
}
```

##

api_server

- 서버에서 넘기는 용도, "chart": result.get("chart"), 부분이 추가된다.

```
return {
...
"draft": result["draft"],
"chart": result.get("chart"),
"updated_params": result.get("updated_params"),
}
```

##

front-end(JS기준)

```
# 출력시
addMessage('agent', data.draft || data.result || data.answer || JSON.stringify(data), data.chart || null);

# addMessage 함수 부분
function addMessage(role, content, imageB64 = null) {
  const empty = document.getElementById('emptyState');
  if (empty) empty.remove();

  const wrap = document.createElement('div');
  wrap.className = `message-wrap ${role}`;

  if (role === 'agent') {
    wrap.innerHTML = `
      <div class="avatar">AI</div>

      # 아래 div 상자가 chart img 부분
      <div class="bubble">${escapeHtml(content)}${imageB64
        ? `<img src="${imageB64}" alt="시뮬레이션 그래프">`
```

```
        : ''
      }</div>`;

  } else {
    wrap.innerHTML = `<div class="bubble">${escapeHtml(content)}</div>`;
  }

  document.getElementById('messages').appendChild(wrap);
  scrollToBottom();
  return wrap;
}
```

폐기될시: 유감

(result 텍스트를 정제하지 않았지만...) chart가 답변에 포함되는 모습

